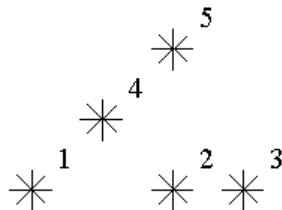


ITMO-2026-Spring-05, продвинутое дерево отрезков и корневая декомпозиция

A.1. Stars

0.5 seconds, 512 megabytes

Astronomers often examine star maps where stars are represented by points on a plane and each star has Cartesian coordinates. Let the level of a star be an amount of the stars that are not higher and not to the right of the given star. Astronomers want to know the distribution of the levels of the stars.



For example, look at the map shown in the figure above. The level of star number 5 is equal to 3 (it's formed by three stars with numbers 1, 2 and 4). And the levels of the stars 2 and 4 are 1. At this map, there is only one star of level 0, two stars of level 1, one star of level 2, and one star of level 3. You are to write a program that will count the amounts of the stars of each level on a given map.

Input

Input contains one or more tests. Each test is described as follows.

The first line of the test contains a number of stars N ($1 \leq N \leq 300\,000$). The following N lines describe the coordinates of stars (two integers X and Y per line separated by a space, $0 \leq X, Y < 10^6$).

There can be only one star at one point of the plane. Stars are listed in ascending order of the Y coordinate. Stars with equal Y coordinates are listed in ascending order of the X coordinate.

The total sum of N of all tests is not greater than 300 000.

Output

Output should contain answers to all tests. Each answer should be described as follows. N lines, one number per line. The first line contains the amount of stars of level 0, the second does the amount of stars of level 1 and so on. The last line contains the amount of stars of level $N-1$.

```
input
5
1 1
5 1
7 1
3 3
5 5
5
1 1
5 1
7 1
3 3
5 5
```

output

```
1
2
1
1
0
1
2
1
1
0
```

Statement
is not
available
in
English
language

B.1. Persistent Array

0.4 секунд, 512 мегабайт

Дан массив (вернее, первая, начальная его версия).

Нужно уметь отвечать на два запроса:

- $a_i[j] = x$ — создать из i -й версии новую, в которой j -й элемент равен x , а остальные элементы такие же, как в i -й версии.
- $\text{get } a_i[j]$ — сказать, чему равен j -й элемент в i -й версии.

Входные данные

Количество чисел в массиве N ($1 \leq N \leq 10^5$) и N элементов массива. Далее количество запросов M ($1 \leq M \leq 10^5$) и M запросов. Формат описания запросов можно посмотреть в примере. Если уже существует K версий, новая версия получает номер $K + 1$. И исходные, и новые элементы массива — целые числа от 0 до 10^9 . Элементы в массиве нумеруются числами от 1 до N .

Выходные данные

На каждый запрос типа `get` вывести соответствующий элемент нужного массива.

входные данные

```
6
1 2 3 4 5 6
11
create 1 6 10
create 2 5 8
create 1 5 30
get 1 6
get 1 5
get 2 6
get 2 5
get 3 6
get 3 5
get 4 6
get 4 5
```

выходные данные

```
6
5
10
5
10
8
6
30
```

Оффлайн версия. Решается за чистую линию dfs-ом.

Statement
is not
available
in
English
language

C.2. Ближайшая большая справа

0.5 секунд⁹, 512 мегабайт

Дан массив a из n чисел. Нужно обрабатывать запросы:

- `set(i, x)` — присвоить $a[i] = x$;
- `get(i, x)` — найти $\min k: k \geq i$ и $a_k \geq x$.

Входные данные

На первой строке длина массива n и количество запросов m . На второй строке n целых чисел — массив a . Следующие m строк содержат запросы. Индексы в массиве нумеруются с 1. Запрос типа `set: "0 i x"`. Запрос типа `get: "1 i x"`.

$$1 \leq n, m \leq 200\,000.$$

$$1 \leq i \leq n.$$

$$0 \leq x, a_i \leq 200\,000.$$

Выходные данные

На каждый запрос типа `get` на отдельной строке выведите k . Если такого k не существует, выведите -1 .

входные данные

```
4 5
1 2 3 4
1 1 1
1 1 3
1 1 5
0 2 3
1 1 3
```

выходные данные

```
1
3
-1
2
```

Делается простейшим одномерным деревом отрезков.

Statement
is not
available
in
English
language

D.2. Перестановки

1.5 секунд⁹, 512 мегабайт

Вася выписал на доске в каком-то порядке все числа от 1 по N , каждое число ровно по одному разу. Количество чисел оказалось довольно большим, поэтому Вася не может окинуть взглядом все числа. Однако ему надо всё-таки представлять эту последовательность, поэтому он написал программу, которая отвечает на вопрос — сколько среди чисел, стоящих на позициях с x по y , по величине лежат в интервале от k до l . Сделайте то же самое.

Входные данные

В первой строке лежит два натуральных числа — $1 \leq N \leq 100\,000$ — количество чисел, которые выписал Вася и $1 \leq M \leq 100\,000$ — количество вопросов, которые Вася хочет задать программе. Во второй строке дано N чисел — последовательность чисел, выписанных Васей. Далее в M строках находятся описания вопросов. Каждая строка содержит четыре целых числа $1 \leq x \leq y \leq N$ и $1 \leq k \leq l \leq N$.

Выходные данные

Выведите M строк, каждая должна содержать единственное число — ответ на Васин вопрос.

входные данные

```
4 2
1 2 3 4
1 2 2 3
1 3 1 3
```

выходные данные

```
1
3
```

Вы умеете решать эту задачу несколькими способами. Любое решение подойдёт.

Statement
is not
available
in
English
language

E.2. Овна

0.7 секунд⁹, 512 мегабайт

На экране расположены прямоугольные окна, каким-то образом перекрывающиеся (со сторонами, параллельными осям координат). Вам необходимо найти точку, которая покрыта наибольшим числом из них.

Входные данные

В первой строке входного файла записано число окон n ($1 \leq n \leq 50\,000$).

Следующие n строк содержат координаты окон

$x_{(1,i)} \ y_{(1,i)} \ x_{(2,i)} \ y_{(2,i)}$, где $\langle x_{(1,i)}, y_{(1,i)} \rangle$ — координаты левого верхнего угла i -го окна, а $\langle x_{(2,i)}, y_{(2,i)} \rangle$ — правого нижнего (на экране компьютера y растёт сверху вниз, а x — слева направо).

Все координаты — целые числа, по модулю не превосходящие $2 \cdot 10^5$.

Выходные данные

В первой строке выходного файла выведите максимальное число окон, покрывающих какую-либо из точек в данной конфигурации. Во второй строке выведите два целых числа, разделенные пробелом — координаты точки, покрытой максимальным числом окон. Окна считаются замкнутыми, т.е. покрывающими свои граничные точки.

входные данные
2 0 0 3 3 1 1 4 4
выходные данные
2 1 2

Сканлайн с деревом отрезков.

F.2. Memory manager

2 seconds🕒, 512 megabytes

One of the key features of the newest operating system Indows II is the new memory manager. It operates on an array of length N and supports three cutting-edge operations:

- `copy(a, b, l)`, which copies an interval of length $[a, a+l-1]$ to $[b, b+l-1]$;
- `sum(l, r)`, which calculates the sum of elements in the $[l, r]$ interval;
- `print(l, r)`, prints elements from l to r , inclusive.

You are developing your own operating system, and you absolutely have to support these mind-blowing operations. Hence, you have to implement your own memory manager.

Input

First line contains an integer N ($1 \leq N \leq 1\,000\,000$), which is the size of the array that your memory manager is going to work with.

Second line contains four integers $1 \leq X_1, A, B, M \leq 10^9 + 10$.

These are used to generate the initial array: $X_1, X_2, \dots, X_N$.

$$X_{i+1} = (A \cdot X_i + B) \bmod M$$

Third line contains an integer K ($1 \leq K \leq 10\,000$), which is the number of operations your memory manager is going to process.

Then there are K lines with operations in the following form:

- `cpy a b l` for the `copy` operation;
- `sum l r` for the `sum` ($l \leq r$) operation;
- `out l r` for the `print` ($l \leq r$) operation.

It is guaranteed that altogether `print` will output no more than 3 000 elements. It is also guaranteed that all operations are valid.

Output

For all `sum` and `print` operations output the result on a separate line.

input
6 1 4 5 7 7 out 1 6 cpy 1 3 2 out 1 6 sum 1 4 cpy 1 2 4 out 1 6 sum 1 6

output

```
1 2 6 1 2 6
1 2 1 2 2 6
6
1 1 2 1 2 6
13
```

In this version of the problem, the parameter K is not so large, which cannot be said about N .

Estimate the time complexity of *persistent* operations. Make its dependency on N smaller.

Garbage collection should not be needed.

Statement
is not
available
in
English
language

G.3. Менеджер памяти

4 секунды🕒, 512 мегабайт

Одно из главных нововведений новейшей операционной системы Indows 7 — новый менеджер памяти. Он работает с массивом длины N и позволяет выполнять три самые современные операции:

- `copy(a, b, l)` — скопировать отрезок длины $[a, a+l-1]$ в $[b, b+l-1]$
- `sum(l, r)` — посчитать сумму элементов массива на отрезке $[l, r]$
- `print(l, r)` — напечатать элементы с l по r , включительно

Вы являетесь разработчиком своей операционной системы, и Вы, безусловно, не можете обойтись без инновационных технологий. Вам необходимо реализовать точно такой же менеджер памяти.

Входные данные

Первая строка входного файла содержит целое число N

($1 \leq N \leq 1\,000\,000$) — размер массива, с которым будет работать Ваш менеджер памяти.

Во второй строке содержатся четыре числа

$1 \leq X_1, A, B, M \leq 10^9 + 10$. С помощью них можно сгенерировать исходный массив чисел $X_1, X_2, \dots, X_N$.

$$X_{i+1} = (A \cdot X_i + B) \bmod M$$

Следующая строка входного файла содержит целое число K

($1 \leq K \leq 200\,000$) — количество запросов, которые необходимо выполнить Вашему менеджеру памяти.

Далее в K строках содержится описание запросов. Запросы заданы в формате:

- `cpy a b l` — для операции `copy`
- `sum l r` — для операции `sum` ($l \leq r$)
- `out l r` — для операции `print` ($l \leq r$)

Гарантируется, что суммарная длина запросов `print` не превышает 3 000. Также гарантируется, что все запросы корректны.

Выходные данные

Для каждого запроса `sum` или `print` выведите в выходной файл на отдельной строке результат запроса.

входные данные

```
6
1 4 5 7
7
out 1 6
сру 1 3 2
out 1 6
sum 1 4
сру 1 2 4
out 1 6
sum 1 6
```

выходные данные

```
1 2 6 1 2 6
1 2 1 2 2 6
6
1 1 2 1 2 6
13
```

Добавьте сборку мусора в своё персистентное дерево (garbage collector ссылочный, или ленивый, или хотя бы `shared_ptr`). Любители `shared_ptr`, возможно, получат TL или ML.

Ленивый аллокатор: стек свободных вершин + если память закончилась, собрать мусор.

«Списочный аллокатора» не получится: нужно заранее выделить список из $10^6 + 1000$ вершин; переопределить `new/delete`.

Statement
is not
available
in
English
language

Н.3. Фаброзавры-дизайнеры

2 секунды, 512 мегабайт

Фаброзавры известны своим тонким художественным вкусом и увлечением ландшафтным дизайном. Они живут около очень живописной реки и то и дело перестраивают тропинку, идущую вдоль реки: либо насыпают дополнительной земли, либо срывают то, что есть. Для того чтобы упростить эти работы, они поделили всю тропинку на горизонтальные участки, пронумерованные от 1 до N , и их переделки устроены всегда одинаково: они выбирают часть дороги от L -го до R -го участка (включительно) и изменяют (увеличивают или уменьшают) высоту на всех этих участках на одну и ту же величину (если до начала переделки высоты были разными, то и после переделки они останутся разными).

Поскольку, как уже говорилось, у фаброзавров тонкий художественный вкус, каждый из них считает, что их река лучше всего выглядит с определенной высоты. Поэтому им хочется знать, есть ли поблизости от их дома место на тропинке, где высота, на их взгляд, оптимальна. Помогите им в этом разобраться.

Входные данные

Первая строка входного файла содержит два числа N и M — длину дороги и количество запросов соответственно ($1 \leq N, M \leq 10^5$). На второй строке содержатся N чисел, разделенных пробелами — начальные высоты соответствующих частей дороги; высоты не превосходят 10^4 по модулю. В следующих M строках содержатся запросы по одному на строке.

Запрос $+ L R X$ означает, что высоту частей дороги от L -й до R -й (включительно) нужно изменить на X . При этом $1 \leq L \leq R \leq N$, а $|X| \leq 10^4$.

Запрос $? L R X$ означает, что нужно проверить, есть ли между L -м и R -м участками (включая эти участки) участок, где дорога проходит точно на высоте X . Гарантируется, что $1 \leq L \leq R \leq N$, а $|X| \leq 10^9$.

Выходные данные

На каждый запрос второго типа нужно вывести в выходной файл на отдельной строке одно слово «YES» (без кавычек), если нужный участок существует, и «NO» в противном случае.

входные данные

```
10 5
0 1 1 3 3 3 2 0 0 1
? 3 5 2
+ 1 4 1
? 3 5 2
+ 7 10 2
? 9 10 3
```

выходные данные

```
NO
YES
YES
```

Что нужно хранить в куске? Можно и проще: сортированный массив, можно хеш-таблицу.

Statement
is not
available
in
English
language

I.3. Count Online

3 секунды, 512 мегабайт

Вам дано множество точек на плоскости.

Нужно уметь отвечать на два типа запросов:

- $? x_1 y_1 x_2 y_2$ — сказать, сколько точек лежит в прямоугольнике $[x_1..x_2] \times [y_1..y_2]$. Точки на границе и в углах тоже считаются. $x_1 \leq x_2, y_1 \leq y_2$.
- $+ x y$ — добавить в множество точку $(x + \text{res} \% 100, y + \text{res} \% 101)$. Где res — ответ на последний запрос вида $?$, а $\%$ — операция взятия по модулю.

Входные данные

Число точек N ($1 \leq N \leq 50\,000$). Далее N точек. Число запросов Q ($1 \leq Q \leq 100\,000$). Далее Q запросов. Все координаты от 0 до 10^9 .

Выходные данные

Для каждого запроса $?$ одно целое число — количество точек внутри прямоугольника.

входные данные

```

5
0 0
1 0
0 1
1 1
1 1
9
? 0 1 1 2
+ 1 2
+ 2 2
? 1 0 2 2
? 0 0 0 0
+ 3 3
? 3 3 3 3
? 4 3 4 3
? 4 4 5 5

```

выходные данные

```

3
3
1
0
0
3

```

На самом деле добавлялись точки $(4, 5)$, $(5, 5)$, $(4, 4)$.

Hint: Корневая по запросам.

Statement
is not
available
in
English
language

J.3. Прямоугольники

0.75 секунд[?], 512 мегабайт

На плоскости задано n прямоугольников, никакие два из которых не имеют общих точек. В каждом прямоугольнике записано целое число.

Скажем, что прямоугольник B лежит дальше прямоугольника A , если левый верхний угол прямоугольника B лежит строго ниже и правее правого нижнего угла прямоугольника A .

Последовательность прямоугольников $R_1, R_2, \dots, R_k$ назовем цепью, если для всех i прямоугольник R_i лежит дальше прямоугольника R_{i-1} . Весом цепи назовем сумму чисел, записанных во входящих в неё прямоугольниках.

Требуется найти цепь прямоугольников с максимальным весом.

Входные данные

Первая строка входного файла содержит число n — количество прямоугольников ($1 \leq n \leq 100\,000$).

Пусть ось x направлена слева направо, а ось y — снизу вверх. Следующие n строк содержат по пять целых чисел — координаты $x_{i,1}, y_{i,1}$ левого нижнего, $x_{i,2}, y_{i,2}$ правого верхнего углов прямоугольника и a_i — число, записанное в прямоугольнике. Координаты не превышают 10^9 по абсолютной величине. Числа, записанные в прямоугольниках, положительные и не превышают 10^9 . Ни один прямоугольник не лежит внутри другого.

Выходные данные

В первой строке выходного файла выведите одно число — максимальный возможный вес цепи прямоугольников. Во второй строке выведите через пробелы номера прямоугольников, образующих такую цепь, в порядке цепи. Если оптимальных решений несколько, разрешается вывести любое из них.

входные данные

```

4
1 1 2 2 6
3 1 4 2 5
0 3 1 4 5
5 1 6 2 4

```

выходные данные

```

10
3 2

```

Ещё один сканлайн.